Architecture and Protocols

Chosen Software Design Pattern: MVVM (Model-View-ViewModel)

Since we are currently learning Flutter, we wanted an architecture that helps us understand how to separate the user interface from the business logic without being overly complicated. We decided to use the MVVM architecture, likely paired with a beginner-friendly state management solution like `Provider`.

- Model: This is where we define our core data structures using standard Dart classes. For instance, we will have an `Asset` model (ticker name, current price) and a `UserProfile` model (containing the 1-7 risk score).
- View: These are the visual screens we designed in Figma (e.g., `DashboardScreen` and `RiskResultScreen`). In Flutter, these will be our UI Widgets. The View's only job is to render the layout and listen for user interactions.
- ViewModel: This will be our state management class (e.g., an `AssetProvider`). It will fetch live market prices, calculate the 1-7 risk score, and notify the View to rebuild when the data changes. This keeps our Flutter UI files clean.

Key Protocols and Interfaces

To make our Flutter app function with real data, we will rely on the following protocols and interfaces:

- REST API over HTTPS: We will use Flutter's `http` (or `dio`) package to make standard GET requests to a third-party financial API (such as Yahoo Finance) to pull live market data.
- JSON Parsing: The data coming from the external APIs will be in JSON format. We will use Dart's built-in `dart:convert` library to parse this JSON into our Model classes.
- OAuth 2.0: For the login and signup flow, we plan to implement OAuth 2.0. This ensures user credentials are handled securely.
- Local Device Storage: To save the user's calculated risk score (1-7) locally on their device, we will use Flutter's `shared_preferences` interface. This allows us to persist data so the user doesn't have to retake the risk test every time they open the app.